Sampling and performing gauge transformations between models
Notes

Clélia – June 2017

Models belonging to the same complexity class can be seen as the same model observed from different points of view, or in different worlds. Mathematically, they are the same model expressed in different bases. These notes give an insight into the different notations that can be used to represent and manipulate models and gauge transformations for mathematical or numerical purposes.

I. MONOMIALS (OR BINARY SPIN VARIABLES) AND MODELS

For instance, consider the model $\mathcal{M}$ on $n = 4$ spins defined by the 3 monomials,

$$\mathcal{M} : \begin{cases} \phi_1 &= s_1 s_2 s_3 \\ \phi_2 &= s_1 s_2 s_4 \\ \phi_3 &= s_2 s_3 \,, \end{cases} \tag{1}$$

and the gauge transformation between the 4 spins variables (s_1, s_2, s_3, s_4) and the 4 spins variables $(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ defined by

$$\mathcal{G} : \begin{cases} s_1 &= \sigma_2 \\ s_2 &= \sigma_1 \sigma_2 \\ s_3 &= \sigma_2 \sigma_3 \\ s_4 &= \sigma_1 \sigma_2 \sigma_3 \sigma_4 \,. \end{cases} \tag{2}$$

The transformation in the new basis $(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ of the model $\mathcal{M}$ gives the model $\mathcal{M}'$ defined by the monomials:

$$\mathcal{M}' : \begin{cases} \phi_1 &= (\sigma_2)(\sigma_1\sigma_2)(\sigma_2\sigma_3) &= \sigma_1\sigma_2\sigma_3 \\ \phi_2 &= (\sigma_2)(\sigma_1\sigma_2)(\sigma_1\sigma_2\sigma_3\sigma_4) &= \sigma_2\sigma_3\sigma_4 \\ \phi_3 &= (\sigma_1\sigma_2)(\sigma_2\sigma_3) &= \sigma_1\sigma_3 \,, \end{cases} \tag{3}$$

obtained by replacing the s_1, s_2, s_3 and s_4 in ϕ_1, ϕ_2 and ϕ_3 by their expression in the basis of the $\{\sigma_i\}$ (see Eq. (2)), and using that the square of a spin variable is always equal to $(+1)$, $\sigma_i^2 = \mathbb{1}$. The graphic representations of the models $\mathcal{M}$ and $\mathcal{M}'$ are displayed in Fig. 1.

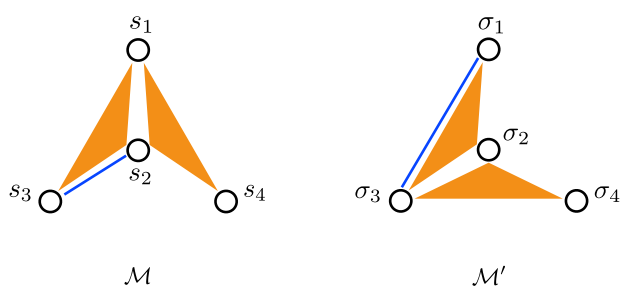

FIG. 1: **Left.** Model $\mathcal{M}$ and $\mathcal{M}'$ defined, respectively, in Eq. (1) and Eq. (3). These models belongs to the same complexity class, being equivalent under the gauge transformation defined in Eq. (2). One can also observe that the permutation of spins $(s_1 = \sigma_2, s_2 = \sigma_3, s_3 = \sigma_1, s_4 = \sigma_4)$ also defines an equivalence relation between these two models: the gauge transformation relating/from one model to another is not always unique.

II. BINARY REPRESENTATION OF MONOMIALS

In Ref. [**?**], we labelled the monomials ϕ using an integer μ [encoded on n bits], $\mu \in \{1, \cdots, 2^n - 1\}$, whose binary representation directly identifies the spins that belong to the set μ. In other words, the spin s_i belongs to

μ if the i-th digit in the base-2 representation of μ is "1". For example, $\phi^7(\boldsymbol{s})$ identifies the three body interaction $\phi^7(\boldsymbol{s}) = s_1 s_2 s_3$ since the binary representation of 7 is $0\cdots0111$, with "1" in the 1rst, 2nd and 3rd positions along the binary string. The model in Eq. (1) is thus identified by $\mathcal{M} = \{\phi^7(\boldsymbol{s}),\, \phi^{11}(\boldsymbol{s}),\, \phi^6(\boldsymbol{s})\}$, whereas the model of Eq. (3) is $\mathcal{M}' = \{\phi^7(\boldsymbol{\sigma}),\, \phi^{14}(\boldsymbol{\sigma}),\, \phi^5(\boldsymbol{\sigma})\}$, and the gauge transformation Eq. (2) is given by the system of 4 monomial equations: $\mathcal{G} = \{s_1 = \phi^2(\boldsymbol{\sigma}), s_2 = \phi^3(\boldsymbol{\sigma}), s_3 = \phi^6(\boldsymbol{\sigma}), s_4 = \phi^{15}(\boldsymbol{\sigma})\}$.

$$\mathcal{G}: \begin{cases} s_1 = \phi^2(\boldsymbol{\sigma}) \\ s_2 = \phi^3(\boldsymbol{\sigma}) \\ s_3 = \phi^6(\boldsymbol{\sigma}) \\ s_4 = \phi^{15}(\boldsymbol{\sigma}) \end{cases} \Rightarrow \mathcal{G}: \begin{cases} \phi^1(\boldsymbol{s}) = \phi^2(\boldsymbol{\sigma}) \\ \phi^2(\boldsymbol{s}) = \phi^3(\boldsymbol{\sigma}) \\ \phi^4(\boldsymbol{s}) = \phi^6(\boldsymbol{\sigma}) \\ \phi^8(\boldsymbol{s}) = \phi^{15}(\boldsymbol{\sigma}) \end{cases} \tag{4}$$

Numerically, the notation with integers encoded in n bits provide a very efficient way to manipulate models (and data) using bit operations. Indeed, the product of two monomials is now given by the XOR bit operation between the two integers identifying these monomials.

III. VECTOR REPRESENTATION

Inspired from the binary representation, the efficiency of working directly with the binary representations.... In a given basis, each monomial can be represented by a vector of n binary elements, corresponding to the coordinates of the monomial in the chosen basis. For instance, in the basis (s_1, s_2, s_3, s_4) the monomials of previous model $\mathcal{M}$ take the form:

$$\phi_1 = \begin{pmatrix}1\\1\\1\\0\end{pmatrix} \qquad \phi_2 = \begin{pmatrix}1\\1\\0\\1\end{pmatrix} \qquad \phi_3 = \begin{pmatrix}0\\1\\1\\0\end{pmatrix} \begin{matrix}\to s_1\\ \to s_2\\ \to s_3\\ \to s_4\end{matrix} \quad , \tag{5}$$

whereas in the new basis $(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ they take the form

$$\phi_1 = \begin{pmatrix}1\\1\\1\\0\end{pmatrix} \qquad \phi_2 = \begin{pmatrix}0\\1\\1\\1\end{pmatrix} \qquad \phi_3 = \begin{pmatrix}1\\0\\1\\0\end{pmatrix} \begin{matrix}\to \sigma_1\\ \to \sigma_2\\ \to \sigma_3\\ \to \sigma_4\end{matrix} \quad . \tag{6}$$

Note that these vectors can also be read as the binary representation of each monomial in the given basis, and that is this notation, the identity monomials $\mathbb{1}$ is a column vectors of 0. The same way we can also defined a gauge transformation by the expression of each element of the initial basis in the arrival basis. For instance, in the example of Eq. (2):

$$s_1 = \begin{pmatrix}0\\1\\0\\0\end{pmatrix} \qquad s_2 = \begin{pmatrix}1\\1\\0\\0\end{pmatrix} \qquad s_3 = \begin{pmatrix}0\\1\\1\\0\end{pmatrix} \qquad s_4 = \begin{pmatrix}1\\1\\1\\1\end{pmatrix} \begin{matrix}\to \sigma_1\\ \to \sigma_2\\ \to \sigma_3\\ \to \sigma_4\end{matrix} \quad . \tag{7}$$

Using this notation, the expression of a monomial can be easily transformed from one basis to another. As we saw earlier, the product operation between monomials is equivalent to perform a xor operation on their binary representations. So that we can write

$$\phi_1 = s_1 \cdot s_2 \cdot s_3 \qquad \longrightarrow \qquad \phi_1 = \begin{pmatrix}0\\\mathbf{1}\\0\\0\end{pmatrix} \otimes \begin{pmatrix}1\\\mathbf{1}\\0\\0\end{pmatrix} \otimes \begin{pmatrix}0\\\mathbf{1}\\1\\0\end{pmatrix} = \begin{pmatrix}1\\0\\0\\0\end{pmatrix} \otimes \begin{pmatrix}0\\1\\1\\0\end{pmatrix} = \begin{pmatrix}1\\\mathbf{1}\\1\\0\end{pmatrix} \begin{matrix}\to \sigma_1\\ \to \sigma_2\\ \to \sigma_3\\ \to \sigma_4\end{matrix} \quad . \tag{8}$$

Note that the xor operator over/on several vectors corresponds to summing the number of 1 on each line a modulo 2 at the end. See for instance the line highlighted in red in Eq. (8). Note that this representation is only putting under the form of vectors what was already done directly on the bits representation of each monomials.

IV. MATRIX REPRESENTATION

The final and simplest way to represent/manipulate models, monomials and gauge transformations, is to use a matrix representation. The matrix representation of a model in a given basis is obtained by combining the vector representation of its monomials in this basis. For instance, in the basis of the (s_i), the matrix representation of the model in Eq. (1) is

$$M = \begin{array}{c} \begin{array}{ccc} \phi_1 & \phi_2 & \phi_3 \end{array} \\ \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \end{array} \begin{array}{l} \\ \to s_1 \\ \to s_2 \\ \to s_3 \\ \to s_4 \end{array} , \tag{9}$$

which is obtained by combining the column vectors of Eq. (5). The gauge transformation corresponding to a change of basis from the (s_i) to the (σ_i) is then defined by the $n \times n$ binary matrix combining the column vectors of the (s_i) written in the basis of the (σ_i). For instance, the gauge transformation Eq. (2) reads:

$$G = \begin{array}{c} \begin{array}{cccc} s_1 & s_2 & s_3 & s_4 \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array} \begin{array}{l} \\ \to \sigma_1 \\ \to \sigma_2 \\ \to \sigma_3 \\ \to \sigma_4 \end{array} , \tag{10}$$

which is obtained by combining the column vectors of Eq. (7). Gauge transformations should describe a bijection between the two basis (s_i) and (σ_i), so that we expect these transformations to be invertible. In other words, any matrix G representing a gauge transformation must necessarily be invertible ($\det G \neq 0$). The new operators, i.e. matrix representation M' of the new model in the basis of the $\{\sigma_i\}$ is finally obtained by multiplying appropriately the two previous matrices: $M' = G\,M$. With our previous example, we obtain:

$$\begin{array}{c} \begin{array}{ccc} \phi_1 & \phi_2 & \phi_3 \\ \downarrow & \downarrow & \downarrow \end{array} \\ \underbrace{\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{M} \end{array} \begin{array}{l} \\ \to s_1 \\ \to s_2 \\ \to s_3 \\ \to s_4 \end{array}$$

$$M' = G\,M = \underbrace{\begin{array}{c} \begin{array}{cccc} s_1 & s_2 & s_3 & s_4 \end{array} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array}}_{G} = \underbrace{\begin{pmatrix} 1 & 0 & 1 \\ \mathbf{1} & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{M'} \begin{array}{l} \\ \to \sigma_1 \\ \to \sigma_2 \\ \to \sigma_3 \\ \to \sigma_4 \end{array} ,$$

which gives back the column vector of Eq. (6). Note that computing this matrix product is [exactly] equivalent to performing the xor operation between the vector representation of the s_i's involved in each monomials ϕ_j (as in the example of Eq. (8)). See for instance the line highlighted in red, which corresponds to the same summation modulo 2 also highlighted in red in Eq. (8), or the first column of M' is obtained by xor operation of the three first column of G.

V. SAMPLING A GAUGE TRANSFORMATION

A. Algorithm with direct GT

Consider a system with n s_i-spin variables. For this system, gauge transformations are any bijective transformations that map the n initial spin variables $\{s_i\}_{1\leq i\leq n}$ to n new spin variables $\{\sigma_i\}_{1\leq i\leq n}$ formed by products of subsets of the initial variables s_i. Mathematically the set of all possible binary variables formed by multiplying subsets of the $\{s_i\}_{1\leq i\leq n}$ spins forms a multiplicative group, Ω_n. The group is said generated by the n elements $\{s_i\}_{1\leq i\leq n}$. And

the gauge transformations correspond to any transformations that maps a set of n generators of Ω_n to a new set of generators.

Complexity of the algorithm grows as $O(2^n)$. Numerically, we find $\sim 0.1 \times 2^n$ (see Fig. 2).

B. Algorithm using the matrix representation of gauge transformations

a. Matrix representation of gauge transformations Consider a system with n s_i-spin variables. For this system, gauge transformations are any bijective transformation that maps the n initial spin variables $\{s_i\}_{1\leq i\leq n}$ to n new spin variables formed by products of subsets of the initial variables s_i. As observed in the previous section, the matrix representation of such transformations correspond to invertible $n \times n$ binary matrices, i.e. the matrices G belonging to the special linear group of degree n over the field F_2; $G \in SL_n(F_2)$ (see [**?**] for a more rigorous proof).

b. Sampling To sample a gauge transformation using the matrix representation, one basic algorithm is to:

1 – sample uniformly a random $n \times n$ binary matrix (each entries is 0 or 1 with probability 1/2);

2 – check if it is invertible.

Given that the number of possible $n \times n$ binary matrices is 2^{n^2} and that the number of these matrices actually invertible is [**?**]

$$|SL_n(F_2)| = \prod_{i=0}^{n-1}(2^n - 2^i) = 2^{n^2}\prod_{i=1}^{n}(1 - \frac{1}{2^i}), \tag{11}$$

which converges rapidly ($n > 5$) to $2^{n^2}\phi(1/2)$, the probability that the matrix randomly sampled is actually invertible is thus $\phi(1/2) \simeq 0.2887880950$. In other terms, on average, 1 matrix is invertible for 3 sampled.

To check if the matrix is invertible, we use a modified version of the Gaussian elimination algorithm (or row reduction algorithm). The program performs operations on the lines of the matrix until it obtains, either a column without leading coefficient ("pivot"), in which case the matrix is not invertible; or the final row echelon form of the matrix with a pivot on each column, in which case the matrix is invertible. In the worst case this algorithm requires $O(n^3)$ boolean operations to check if a matrix is invertible.

c. Speed Written in C++ and run on a personal machine [to specify] we obtained the following performances: see Table I.

C. Combining the binary representation and the matrix representation of a gauge transformation

Consider a $m \times n$ binary matrix whose columns $C[i]_{\{0\leq i<m\}}$ are given by single integers $\phi_i \in [1; 2^n - 1]$, such as the definition of the GT matrix G in ... and the definition of a model in ... . For the Gaussian elimination algorithm we want to do operation on the lines of the matrix. The transpose matrix ${}^tG$ is the matrix whose rows are the for which each row $L[i]$ is equal to the column $C[i]$ of the matrix G. i.e. whose rows are given by the bit representation of integers $\phi[i]$.

Using that, if ${}^tG$ is invertible than G is also invertible (as $det({}^tG) = det(G)$), we can perform the Gaussian elimination on ${}^tG$ to determine if it is invertible.

This algorithm is identical to the previous one on the principle, but instead of working with a binary matrix, we replaced it by a vector of integers in which each integers is the values of the binary column.

Each column vector

To sample :

1 – sample a vector of n integers uniformly distributed over $[0, 2^n - 1]$ (we work with 64 bits integers);

2 – check if the corresponding binary matrix is invertible, working directly on the bits.

To check if the matrix is invertible, we use the same modified Gaussian elimination algorithm. In principle this algorithm is identical to the previous one, but operations are now performs directly on the bits. Each row of the matrix is now defined by a single integer, the binary elements of the row being the bit representation of the given integer. So that the sum modulo 2 of two rows for instance is now simply a xor operation between the two integers defining the [corresponding] rows. In the worst case, the algorithm now requires up to $O(n^3)$ bit operations.

Note that, any vector with an entry 0 corresponds to a matrix with a column of 0's, which is necessarily not invertible. Such vectors will always be rejected by the algorithm; we remove them from the start/sampling by sampling the vector elements directly in $[1, 2^n - 1]$ instead of $[0, 2^n - 1]$.

Program	$n = 5$	$n = 10$	$n = 15$	$n = 30$	$n = 64$	$n = 100$
Direct GT	3.2 s	90 s	2900 s	X	X	X
Boolean Matrix	1.88 s	8.3 s	21.0 s	117 s	900 s	3100 s $\simeq$ 52 min
Int64 Vector	2.0 s	5.2 s	9.0 s	25.7 s	90.4 s	X

TABLE I: Estimated times to sample 10^6 gauge transformations for a system with n spin variables on a personal machine [to specify]. Times of the cases left empty haven't been computed; the "X" symbols means that the program cannot compute gauge transformation for this value of n.

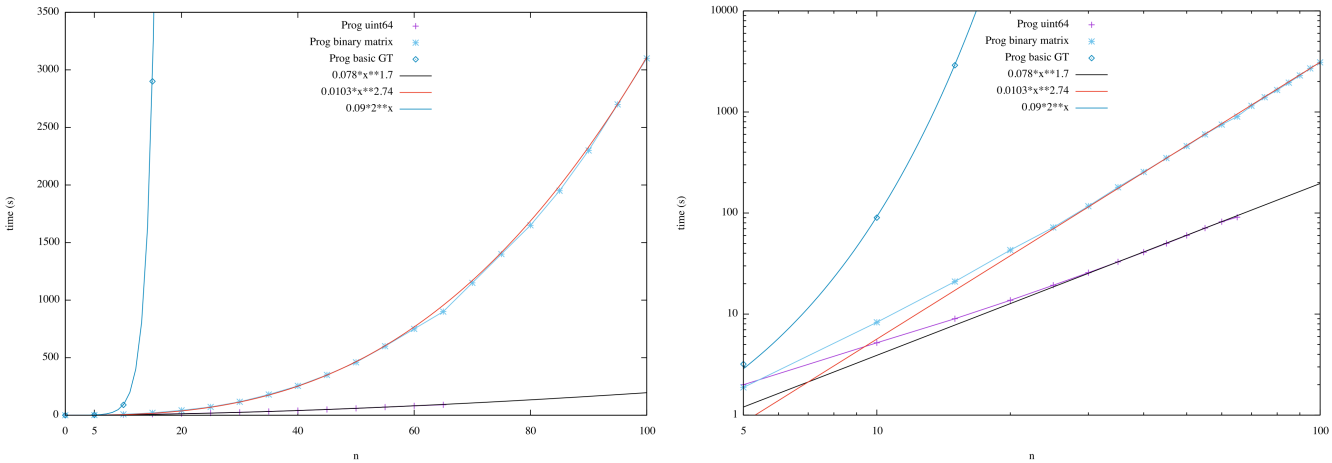

FIG. 2

VI. GAUGE TRANSFORMATION OF A MODEL

Depending on what is more convenient, we can also perform operations on the transposed matrices, using that ${}^{t}M\,{}^{t}G = {}^{t}M'$.